
From ZERO to Hero in Cybersecurity AI

A Super-Friendly Tutorial for Absolute Beginners

No Prior Knowledge Needed. We Start From Scratch.

Challenge: SIH-26153

AI-based Network Attack Forecasting from Network Traffic Data

*“Dear reader, if you have never written a line of code, never heard of an IP address, and think AI is magic — this book is for you. By the last page, you will understand how to build an AI that can **dream about the future of a computer network** and warn us before a hacker strikes. I will hold your hand the whole way. Let’s begin together.”*

— Your Teacher

What We Will Learn Together

Chapter 0: The Absolute Basics (What Even Is a Network? What Is AI?)
Chapter 1: How Messages Travel on the Internet (Network Telemetry)
Chapter 2: How Hackers Attack Step-by-Step (Lifecycles & Datasets)
Chapter 3: Teaching AI to Imagine the Future (World Models)
Chapter 4: Making AI Explain Its Decisions (Interpretability)
Chapter 5: Turning Your AI Into a Real Product (Systems)

Contents

How to Use This Book	2
1 Chapter 0: The Absolute Basics	
What Even Is a Network? What Is AI? Let's Start From Zero.	3
1.1 0.1 What Is a Computer Network? Roads and Houses.	3
1.2 0.2 What Is Data? Letters, Forms, and Report Cards.	3
1.3 0.3 What Is Cybersecurity? Guards, Burglars, and Alarms.	4
1.4 0.4 What Is Artificial Intelligence and Machine Learning? Teaching a Child.	4
1.5 0.5 What Is a Feature Vector? Ingredients List.	4
1.6 0.6 What Do Probability, P , and Those Maths Symbols Mean?	5
2 Chapter 1: How Messages Travel on the Internet	
Network Telemetry & Feature Engineering	6
2.1 1.1 The Big Picture: The Postal System of the Internet	6
2.2 1.2 IP Addresses and Ports: House Numbers and Apartment Doors	6
2.3 1.3 The IP Header: Writing on the Outside of the Envelope	7
2.4 1.4 The TCP Header: The Rules of a Polite Phone Call	7
2.5 1.5 UDP: The Postcard (Quick but No Guarantee)	8
2.6 1.6 From Many Letters to One Summary: What Is a Flow?	9
2.7 1.7 NetFlow and IPFIX: The Standard Way to Write Report Cards	9
2.8 1.8 Two Levels Are Better Than One: Flows AND Packets	10
2.9 1.9 Hands On: Reading Packets with Python (Line by Line)	10
2.9.1 The Whole Pipeline in One Line	11
2.10 1.10 Cutting Time into Slices: Windows and Discretisation	11
2.10.1 How to Define a Window	12
2.10.2 Choosing How Big a Slice Should Be (Goldilocks)	12
3 Chapter 2: How Hackers Attack Step-by-Step	
Cybersecurity Attack Lifecycles & Datasets	14
3.1 2.1 The Attacker's Playbook: MITRE ATT&CK (The Big Map)	14
3.1.1 The Full List of Tactics (In Order They Usually Happen)	15
3.1.2 Some Real Technique IDs (So You Sound Like a Pro)	16
3.1.3 Why Mapping to ATT&CK Makes Your Project Stronger	16
3.2 2.2 Your Practice Papers: The Datasets	16
3.2.1 The Two Datasets Named in Your Challenge	17
3.2.2 Other Datasets You Might Hear About	17
3.2.3 What One Row of CIC-IDS Looks Like (One Report Card)	17
3.3 2.3 The Silent Cheater: Temporal Data Leakage	18
3.3.1 Four Ways Leakage Sneaks In (and How to Block Each)	18
3.3.2 The Honest Rules (Pin This to Your Wall)	19
4 Chapter 3: Teaching AI to Imagine the Future	
AI Dynamics & Sequence Models (The World Model)	20
4.1 3.1 What Is a World Model? A Weather Forecast for Hackers.	20
4.1.1 The One Equation That Says It All (and What It Means)	20
4.1.2 The Three Body Parts of a World Model	20
4.1.3 How We Train It (Without Cheating)	20
4.2 3.2 Why Normal AI Forgets: And How We Fix It with Memory	21
4.2.1 The Problem with a Forgetful Friend	21

4.2.2	The Simple RNN: The Basic Memory Cell	21
4.2.3	The Whisper Game Problem: Vanishing Gradients	22
4.3	3.3 LSTM: The Smart Notebook with Gates (Doors)	22
4.4	3.4 GRU: The Lighter Backpack	23
4.5	3.5 Transformers and Self-Attention: The Spotlight	23
4.5.1	Why Another Model If LSTMs Work?	23
4.5.2	The Spotlight Analogy	24
4.5.3	Multiple Spotlights and Page Numbers	24
4.5.4	The Transformer Block = LEGO Brick	25
4.6	3.6 Dreaming K Steps Ahead: The Rollout	25
4.6.1	Teacher Forcing vs. Real Dreaming	25
4.7	3.7 Graph Neural Networks: The Social Network of Computers	25
4.7.1	Gossip as Maths: Message Passing	26
4.7.2	GCN in One Line (Don't Fear It)	26
4.7.3	Time + Graph = Spatio-Temporal	26
5	Chapter 4: Making AI Explain Itself	
	Interpretability & Explainable AI (XAI)	28
5.1	4.1 Why "Trust Me, I'm AI" Is Not Enough	28
5.2	4.2 SHAP: Fairly Sharing Credit (The Pizza Party)	28
5.2.1	The Pizza Party That Explains Everything	28
5.2.2	The Additive Sentence	28
5.2.3	How to Read a SHAP Value (Like Reading a Report Card)	29
5.2.4	SHAP in Python (You Don't Need to Code This From Scratch)	29
5.3	4.3 Attention Weights: Peeking Over the AI's Shoulder	30
5.3.1	What Are They?	30
5.3.2	How to Get Them in Code	30
5.3.3	The Heatmap (The Picture Judges Remember)	30
6	Chapter 5: Turning It Into a Real Product	
	Deployment, Benchmarking & Systems	32
6.1	5.1 Building an Offline Product (No Internet Needed)	32
6.1.1	The Six Pieces You Must Deliver (The Check-list)	32
6.1.2	The Assembly Line (How Data Flows Through Your Product)	32
6.2	5.2 How Do We Know It's Good? Measuring with Honest Numbers	33
6.2.1	Your Opponent: The Simple Baseline	33
6.2.2	The Four Numbers That Matter (Meet the Confusion Matrix)	33
6.2.3	Extra Scores for Forecasting (You Are a Forecaster, Remember)	34
6.2.4	The Honest Benchmark Recipe (Do This Exactly)	34
6.3	5.3 The Full Journey: Your 10-Step Roadmap	34
	A Final Word From Your Teacher	36
	Appendix A: Glossary	37
	Appendix B: Quick Reference	38
	Appendix C: The Five Forecast Stages	39

How to Use This Book (Please Read This First!)

Hello! Before we learn anything, let me tell you **how** to read this book so you don't get confused or scared.

1. Every hard word is explained the FIRST time it appears. When you see a box that says **Word Bank**, that is me stopping to say: “Hey, this is a new word — here is what it means in simple English.” You never need to Google anything while reading.

2. Every idea starts with a story or a picture. Your brain loves stories. So before I show you a scary formula, I will say “Imagine you are mailing a letter...” Once you get the picture, the formula will feel friendly, not scary.

3. Boxes have meanings. Learn them once:

- **Word Bank** — a new term, explained like you are 10 years old.
- **Imagine This** — close your eyes and picture an everyday scene.
- **Story Time** — a little tale to make the idea stick in memory.
- **Key Idea** — the one thing you **MUST** remember from this section.
- **Let's Work Through an Example** — we solve something together, step by step, with numbers.
- **Your Turn** — a tiny exercise for you. No grading! Just to check you got it. Try with pen and paper.
- **Quick Recap** — a 3-line summary before we move on.
- **Common Mistake** — the trap every beginner falls into.
- **Teacher's Tip** — how this connects to your actual project.

4. You do NOT need to be good at maths. Where maths appears, I will tell you what every symbol means in plain words, and what the equation is *trying to say*. You can understand the story without memorising the maths.

5. Read with a pencil. Underline, scribble “aha!” in the margins, draw little hearts. Active reading beats passive reading every time.

If you are ready, turn the page. Chapter 0 is waiting — and it assumes you know *literally nothing*. Perfect place to start.

1. Chapter 0: The Absolute Basics

What Even Is a Network? What Is AI? Let's Start From Zero.

This chapter is for the true beginner. If you already know what an IP address and machine learning are, you can skim it. If not, welcome — we are going to build your foundation block by block, with zero shame and lots of pictures in your mind.

1.1 0.1 What Is a Computer Network? Roads and Houses.

Imagine This — Picture It in Your Mind

Close your eyes and picture your town. There are **houses** (computers, phones, servers). There are **roads** (wires and Wi-Fi) connecting them. There are **letters and parcels** (messages) travelling on those roads from one house to another. That whole town — houses plus roads plus the system for sending letters — is a **computer network**.

The **Internet** is just a *really big* town where the houses are all over the world and the roads are a mix of cables under the ocean and radio waves through the air.

That is it. When we say “network traffic,” we just mean “all the letters travelling on all the roads at a given time.” When we say “network security,” we mean “making sure bad letters don’t cause harm, and that no one sneaks into a house they shouldn’t.”

Word Bank — New Word Alert!

Network = a group of connected computers that can send messages to each other.

Internet = the biggest network in the world, connecting billions of devices.

Traffic = all the messages moving around the network right now.

Device / **Host** = one house in the town (your laptop, a server, a phone).

1.2 0.2 What Is Data? Letters, Forms, and Report Cards.

Data just means “information written down so a computer can read it.”

Think of three levels:

- **Raw data** = the actual letter inside the envelope. The words “Hi, please send me the file cats.jpg”.
- **Features** = a *summary* someone made from the raw data so it is easier to compare. Like a report card: instead of reading every homework page, you see “Maths: 85, Science: 90.”
- **Label** = the answer written by a teacher: “This letter is **safe**” or “This letter is **dangerous**”.

Key Idea — Remember This!

Features are the clues we give to the AI.

Labels are the correct answers we use to teach the AI.

A **dataset** is a big collection of clues + answers, like a big folder of past exam papers with answer keys.

1.3 0.3 What Is Cybersecurity? Guards, Burglars, and Alarms.

Story Time — An Analogy to Remember

Imagine a museum. It has many rooms (computers), doors (ports — we will learn this!), and guards (security software). A burglar wants to steal a painting (data). They might: look through windows first (scan), try a back door (break in), sneak from room to room (lateral movement), call a friend outside for instructions (command and control), and finally carry the painting out (exfiltration).

Cybersecurity is the job of stopping that burglar — not after the painting is gone, but *as early as possible*, ideally while they are still looking through the windows.

Old security worked like a photo album: “If you see THIS exact face, it’s a burglar.” But burglars change faces. Modern AI security is like learning the *behaviour* of burglars — the *pattern* of how they move — so you can spot even a new burglar you have never seen before.

1.4 0.4 What Is Artificial Intelligence and Machine Learning? Teaching a Child.

Word Bank — New Word Alert!

Artificial Intelligence (AI) = making computers do things that normally need human intelligence (recognising, deciding, predicting).

Machine Learning (ML) = a way to do AI where we don’t write fixed rules, but instead *show the computer many examples* and let it *learn the pattern* itself. Like teaching a child to recognise cats by showing 1000 cat photos, not by writing “cats have pointy ears.”

How does ML work? In super simple terms:

1. You collect a **dataset**: many examples (features) with correct answers (labels).
2. You choose a **model**: a maths recipe with knobs you can turn. At first the knobs are random, so the model’s guesses are terrible.
3. You **train**: show the model one example, let it guess, tell it “you were wrong by this much,” and it turns its knobs a tiny bit to be less wrong. Do this millions of times.
4. You **test**: show it *new* examples it has never seen and see if it still guesses well. If yes, it *learned*, not memorised.
5. You **deploy**: use it in the real world to make predictions.

Imagine This — Picture It in Your Mind

Think of baking. Your first cake is awful. You taste it, note “too much salt,” adjust the recipe, bake again. After 100 cakes, you are a good baker. The “tasting and adjusting” is training. The “try a new recipe on a friend” is testing. That is literally all machine learning is.

1.5 0.5 What Is a Feature Vector? Ingredients List.

A **vector** is just a *list of numbers in a fixed order*. A **feature vector** is the list of clues for one example.

Example: one network conversation (one “flow”) might become:

[80, 421, 0.04, 3, 0.98]

In our code this means: destination port 80, 421 bytes, duration 0.04 seconds, 3 packets, and some timing score 0.98. The *order* matters — position 1 is always the port, position 2 always the bytes, etc. The AI reads this list like you read ingredients: flour, sugar, eggs ...

1.6 0.6 What Do Probability, P , and Those Maths Symbols Mean?

You will see things like $P(A | B)$. Do not panic. Here is the decoder ring:

- $P(X)$ means “the probability (chance) that X happens.” $P = 0$ means never, $P = 1$ means always, $P = 0.8$ means 80% chance.
- $P(A | B)$ means “the probability of A *given that* B happened.” Like “chance of rain GIVEN that clouds are dark.”
- S_t means “the state of the network at time t ” (snapshot at minute t). S_{t+1} is the next minute.
- $\sigma(x)$ (sigmoid) = a squishy function that takes any number and squishes it to between 0 and 1, like a probability.
- $\tanh(x)$ = similar, but squishes to between -1 and 1 .
- $\odot$ = multiply each item in one list with the matching item in another list (element-by-element).
- W and b = knobs the AI turns during training (weights and biases). W is like “how strongly to listen to each clue,” b is “a small nudge.”

You don’t need to calculate these by hand. Just know the *story* each equation tells: “combine the old memory with the new clues, decide what to keep, what to forget, and what to output.”

Quick Recap — What We Just Learned

You now know: networks are towns with roads; traffic is letters on roads; data = written-down information; features = summaries; labels = answers; AI learns by tasting and adjusting like a baker; a feature vector is an ingredients list; P means chance. You are ready for Chapter 1!

2. Chapter 1: How Messages Travel on the Internet

Network Telemetry & Feature Engineering

Telemetry sounds scary. It just means “measuring from far away.” A weather station that radios temperature to your phone is telemetry. Here, *network telemetry* means “measuring what is happening on the network and sending those measurements to our AI.”

2.1 1.1 The Big Picture: The Postal System of the Internet

Imagine This — Picture It in Your Mind

You want to send a birthday gift to your friend. What do you do?

You put the gift (the **actual message**, like “Please show me cats.jpg”) inside a **small box** and write “Deliver to Room 80, Handle with care”. That small box goes inside a **bigger box** with “From: House 1.2.3.4, To: House 5.6.7.8”. That bigger box goes inside an even bigger **mail truck envelope** for the road.

At your friend’s house, they unwrap layer by layer: truck envelope off, then big box, then small box, until the gift appears.

The Internet does EXACTLY this. We call it **encapsulation**: each layer wraps the layer above. And **decapsulation** is unwrapping at the other end.

We name those layers like floors of a building:

Floor 5 — Application: what you actually want (web page, email, file)

Floor 4 — Transport: how the chat is managed (TCP or UDP)

Floor 3 — Network: where it goes (IP addresses)

Floor 2 — Data Link: the local road (Ethernet / Wi-Fi)

Floor 1 — Physical: actual electricity / radio waves on the wire

Each floor adds its own label (header). The Transport letter goes inside the Network parcel which goes inside the Data-Link truck.

Key Idea — Remember This!

For catching hackers, you will spend 95% of your time on just **two floors**:

- **Network (IP)** = the house addresses (who sent, who receives).
- **Transport (TCP/UDP)** = the apartment numbers and chat rules (which door, and is the conversation polite and reliable?).

Everything called a “packet” or “flow” lives in these headers.

2.2 1.2 IP Addresses and Ports: House Numbers and Apartment Doors

This confuses everyone at first. Let’s make it crystal clear.

Word Bank — New Word Alert!

IP address = the street address of a house (computer) on the Internet.

Example: 192.168.1.10 or 142.250.182.14 (Google). Two houses can't have the same address if they want to receive mail.

Port = the apartment/door number *inside* a house. One house can have many doors for different services.

Common doors: 80 = web, 443 = secure web, 22 = secure login (SSH), 53 = address book lookup (DNS).

Protocol number = which language the letter inside is written in: 6 = TCP (reliable chat), 17 = UDP (quick postcard), 1 = ICMP (ping / "are you there?").

So when you see 192.168.1.10:52184 -> 142.250.182.14:443 with TCP, read it as: "From house 192.168.1.10, door 52184, to Google's house at door 443 (secure web), speaking TCP."

Your Turn! — Try It Yourself (No Pressure)

Your computer right now has an IP address. On Windows type `ipconfig`, on Mac/Linux type `ifconfig` in the terminal. Can you find it? What does it start with? Now open a browser and go to `google.com`. You just sent packets to port 443. Magic!

2.3 1.3 The IP Header: Writing on the Outside of the Envelope

Every letter (IP packet) has writing on the outside called a **header**. It is usually 20 bytes long (tiny!). Here is what is written there, in plain English:

- **Source IP** — who sent it. ("From:")
- **Destination IP** — who gets it. ("To:")
- **Time To Live (TTL)** — a countdown number, like "this letter may pass through at most 64 post offices. Each office subtracts 1. If it hits 0, throw it away." Weird TTL jumps hint at **spoofing** (someone faking the From address) or a strangely long route.
- **Protocol** — which inner language (6 = TCP, 17 = UDP).
- **Flags + Fragment info** — "is this letter torn into pieces?" Big letters sometimes get torn into **fragments** to fit through a small mail slot. Clever hackers tear their bad letters into tiny pieces so no single piece looks bad. This is an **evasion trick**.
- **Total Length** — how big the whole thing is, in bytes.

Story Time — An Analogy to Remember

Fragment trick — the scattered puzzle: A burglar wants to smuggle a long message past a guard who checks every letter. So they tear the message into one word per letter and mail them separately. No single letter looks suspicious. But together, they form the plan. That is why we need **packet-level** clues, not just summaries — to catch the scattered puzzle.

2.4 1.4 The TCP Header: The Rules of a Polite Phone Call

TCP is the **polite, reliable** way to talk. If IP is the envelope, TCP is the *conversation style* inside. It promises: "Your words will arrive, in order, without mistakes. If not, I will resend."

It does this with a small header that rides inside the IP envelope:

- **Source Port / Destination Port** — which doors (see above).

- **Sequence Number & Acknowledgment Number** — page numbers for your chat. “I sent pages 1-10, did you get them?” “Got 1-10, send 11 onwards.”
- **Flags** — tiny single-word instructions. These are GOLD for catching hackers. Let’s learn them like verbs:
 - **SYN** = “Hello, can we talk?” (synchronize)
 - **ACK** = “Got it!” (acknowledge)
 - **FIN** = “Goodbye, I’m done.” (finish)
 - **RST** = “Go away, stop talking!” (reset / abort)
 - **PSH** = “Deliver this NOW, don’t wait.” (push)
 - **URG** = “This is urgent!” (urgent)
- **Window Size** — “I can accept this many more words before my desk is full.” Weird window sizes are a fingerprint of *port scanners* and hacking tools.

The TCP Handshake — two people meeting for the first time:

1. You: “Hi, can we talk?” (**SYN**)
2. Them: “Sure, and can *I* also talk to you? Got your hello.” (**SYN** + **ACK**)
3. You: “Great, got your reply, let’s chat!” (**ACK**)

Every normal chat starts exactly like this. Three messages. Always.

Now: a **port scanner** is like a burglar who knocks on *every door* of a building (port 1, port 2, port 3 ... up to 65535) going “Hello? Hello? Hello?” but *never waits for the full hello back*. They just send tons of SYN and vanish. That pattern — *many SYNs, very few ACKs* — is super easy for our AI to spot. That is a **feature** we will use.

2.5 1.5 UDP: The Postcard (Quick but No Guarantee)

UDP is TCP’s laid-back cousin. It just throws a postcard and hopes it arrives. No hello, no confirmation, no resending. It has only four things: from-port, to-port, length, and a tiny check.

It is perfect for things where speed beats perfection: looking up an address (DNS), voice calls, video. But because it doesn’t say hello, it is easy to **fake**. Attackers love sending fake postcards pretending to be someone else (“DNS amplification” attacks). Postcards are hard to trace.

Quick Recap — What We Just Learned

So far: Internet = postal system with layers. IP = house address. Port = door number. TCP = polite phone call with SYN/ACK/FIN. UDP = postcard. Flags are verbs that reveal hacker behaviour.

2.6 1.6 From Many Letters to One Summary: What Is a Flow?

Imagine This — Picture It in Your Mind

Imagine you are a teacher watching a playground. You could write down *every single word* every kid says — that is **packet-level**. Or you could write one summary per *conversation*: “Asha and Ben talked for 2 minutes, exchanged 5 messages, Asha spoke 3 times.” That summary is a **flow**.

Flows are summaries. They are MUCH smaller and they already tell you the story.

Key Idea — Remember This!

A **flow** = one summary line for all packets that share the same **5-tuple** (the ID card of a conversation):

(who-sent-IP, who-sent-port, who-got-IP, who-got-port, protocol)

We usually combine both directions (you-to-me and me-to-you) into one **bidirectional flow** so the summary is complete.

What does one flow record contain? Think of it like a report card for that conversation:

Table 1: A Flow Record Is a Report Card for One Conversation

<i>Group</i>	<i>What it tells you (in plain words)</i>
Who	From which house/door to which house/door, which language (TCP/UDP)
When	When it started, when it ended, how long it lasted
How much	How many bytes and packets each side sent
How fast	Bytes per second, packets per second (throughput)
Rhythm	Time <i>between</i> packets: fastest gap, slowest gap, average gap
Size	Smallest packet, biggest packet, average packet size
Verbs	How many SYN, ACK, PSH, RST flags were seen
Pauses	How long the chat was active vs. idle

These numbers are the **features** your AI will learn from. Instead of reading every word, the AI reads the report card — and that is enough to tell normal chats from hacker chats, most of the time.

2.7 1.7 NetFlow and IPFIX: The Standard Way to Write Report Cards

Routers (the post offices) don’t keep every letter. They write *report cards* (flows) and send them to a collector. The language for writing those report cards has two versions:

- **NetFlow** (made by Cisco) = the original report-card format. Fixed fields, like a printed form.
- **IPFIX** (the newer, official standard) = a *customisable* form. You can add any field you want via **templates**: “also include TCP flags, also include app name.” For security this is wonderful — we can ask the post office to include exactly the clues we need.

For your project, just remember: NetFlow/IPFIX = the system that *creates and exports* flow summaries. When you load a CSV with 80 columns, you are reading exported flows.

2.8 1.8 Two Levels Are Better Than One: Flows AND Packets

- **Flow-level clues** (the report card) catch *loud* attacks: floods (too many packets), exfiltration (too many bytes out).
- **Packet-level clues** (reading actual envelopes) catch *sneaky* attacks: weird TTL, tiny fragments, strange window sizes that look normal in the report card but are odd up close.

Your challenge says: use **both** levels. Two eyes see more than one. A flow that looks normal might still have packet-level oddities that give the hacker away.

Quick Recap — What We Just Learned

Chapter 1 recap: Networks are postal towns. IP = house, port = door, TCP = polite call with verbs (SYN/ACK/FIN/RST), UDP = postcard. A **flow** is a report card summarising one conversation (5-tuple). NetFlow/IPFIX exports those report cards. Use flow-level for loud attacks, packet-level for sneaky ones.

2.9 1.9 Hands On: Reading Packets with Python (Line by Line)

You might be thinking: “Do I need to be a coder?” For this project, a little Python is enough, and I will explain every line like you have never coded before.

Word Bank — New Word Alert!

Python = a human-friendly programming language. Like giving written instructions to a very obedient robot.

Library = a pre-written bundle of instructions someone else made so you don’t start from scratch. Like a recipe book.

PCAP file = a file that contains saved network traffic — a box of saved letters you can open and study without being on the live network.

Scapy = a Python library that can open, read, and even create packets. Pure Python, great for learning.

PyShark = another library that uses Wireshark (a famous network tool) under the hood. Faster for huge files.

Your first packet reader — explained line by line:

```

1 from scapy.all import rdpcap, TCP, IP      # 1. Borrow tools from the
   rdpcap recipe book
2
3 packets = rdpcap("traffic.pcap")          # 2. Open the saved box of
   letters. Now 'packets' is a list.
4
5 for pkt in packets[:5]:                    # 3. Look at just the first
   5 letters, one by one.
6     if IP in pkt and TCP in pkt:          # 4. Only care about letters
   that have BOTH an IP envelope AND a TCP letter inside.
7         ip = pkt[IP]                      # 5. Pull out the IP
   envelope to read it.
8         tcp = pkt[TCP]                    # 6. Pull out the TCP letter
   to read it.
9     print(f"{ip.src}:{tcp.sport} -> {ip.dst}:{tcp.dport} "
10           f"flags={tcp.flags} len={len(pkt)} ttl={ip.ttl}")
11                                           # 7. Print: "From house:door
   -> To house:door flags
   =... size=... ttl=..."

```

What this does: It opens a capture file, picks the first 5 TCP packets, and prints who sent what to whom, with which flags, how big, and what TTL. That is literally the raw material of your whole project.

Teacher's Tip — For Your Project

Do you need to parse packets yourself? If your dataset is already a CSV of flows (like CIC-IDS, which we meet in Chapter 2), you can *skip* parsing and go straight to the next section. Only parse raw PCAPs when you want packet-level detail or when your data is a PCAP. Parsing is the slowest step — always **save (cache)** your result so you don't re-parse every time you run your code. Think of it as photocopying the letters once and reusing the copies.

2.9.1 The Whole Pipeline in One Line

PCAP file $\xrightarrow{\text{read each letter}}$ rows of packets $\xrightarrow{\text{group by 5-tuple}}$ rows of flows
 $\xrightarrow{\text{clean up numbers}}$ feature table (matrix)

Each *row* = one flow (one conversation). Each *column* = one clue (bytes, packets, flag counts ...). This table is what the AI eats. Like a spreadsheet where the AI reads every row.

Your Turn! — Try It Yourself (No Pressure)

If you have Python installed, try installing Scapy (`pip install scapy`) and running the code above on any small PCAP (you can download free ones from [wireshark.org](https://www.wireshark.org)). If you don't have a PCAP yet, that's fine — just read the code again and explain to a friend what line 4 does in your own words. Teaching is the best way to learn.

2.10 1.10 Cutting Time into Slices: Windows and Discretisation

A single flow is one conversation. But a **hacker attack** is a *movie* with many scenes: first they look around, then they break in, then they sneak around, then they steal. To see the movie, you

must cut time into **slices** (windows) and look at the whole picture inside each slice.

Imagine This — Picture It in Your Mind

Imagine a very long security camera recording. You can't watch it all at once. So you cut it into 1-minute clips: Clip 1 (0:00–1:00), Clip 2 (1:00–2:00), ... For each clip you write a summary: "2 people entered, 1 carried a bag out."

Those clips are **windows**. The summary of each clip is the **state** your AI will see at that time step. The AI then watches the *sequence* of clips to learn the story.

Key Idea — Remember This!

Windowing = cutting continuous time into **discrete (separate) slices** so we can feed a *sequence of snapshots* to the AI.

Without windows $\Rightarrow$ no sequence $\Rightarrow$ no story $\Rightarrow$ no World Model. Windows are how we turn time into something the AI can learn from.

2.10.1 How to Define a Window

Pick a size w (say, 60 seconds) and a slide s (how far to move each time).

Window number k covers time $[k \times s, k \times s + w)$. Two styles:

- **Tumbling windows:** $s = w$. Like cutting a cake with no overlap: [0–60), [60–120), [120–180) ... No piece shares crumbs.
- **Sliding windows:** $s < w$. Like overlapping photos: [0–60), [30–90), [60–120) ... More photos, smoother movie, but neighbouring photos look similar.

Word Bank — New Word Alert!

Discretisation = turning smooth, continuous time (like "3:17:04.392") into counted buckets (like "the 3:17 bucket"). Once time is bucketed, we can count things per bucket and the AI can handle it.

2.10.2 Choosing How Big a Slice Should Be (Goldilocks)

- **Too tiny** (e.g., 1 second) $\Rightarrow$ each slice has almost nothing, noisy, and you get a million slices. Hard to see the big picture.
- **Too huge** (e.g., 1 hour) $\Rightarrow$ a whole attack fits in one slice, so you can't see the stages. Blur.
- **Just right** = match the *speed of what you want to catch*. A port scan is fast (seconds). Data theft is slower (minutes). Try a few sizes and see which works best on your data. In your project, try 30s or 60s and compare.

Common Mistake — Don't Do This!

The Sneaky Cheating Trap with Windows:

When you label your windows using the answer key (attack timeline), never cut a *single continuous attack* in half with one half in training and the other half in testing. That lets the AI "peek" at the future. Always split by *time*: first 70% of the movie for training, last 30% for testing, and keep whole attacks together. We will meet this again as "data leakage" in Chapter 2.

Quick Recap — What We Just Learned

Chapter 1 recap in one paragraph: We cut the long movie of network traffic into window-slices, summarise each slice (flows), and turn those summaries into a neat table of numbers. That table is the AI's view of the world. Next, we learn what the *bad guys* actually do inside that movie.

3. Chapter 2: How Hackers Attack Step-by-Step

Cybersecurity Attack Lifecycles & Datasets

3.1 2.1 The Attacker's Playbook: MITRE ATT&CK (The Big Map)

Before you can *predict* a burglar, you must understand how burglars think. Security experts around the world collected every trick real attackers have ever used and wrote them into a giant, free, constantly updated playbook called **MITRE ATT&CK**. Think of it as the “encyclopaedia of hacking steps.”

Word Bank — New Word Alert!

MITRE = a non-profit research group in the USA that made the playbook.

ATT&CK = stands for **A**dversarial **T**actics, **T**echniques, and **C**ommon **K**nowledge.

Tactic = the *WHY*: the attacker's goal at this stage. (“Why are they doing this?” — e.g., “to get inside.”)

Technique = the *HOW*: the specific trick to achieve that goal. (“How do they do it?” — e.g., “by exploiting a bug in the website.”)

Sub-technique = an even more specific version of a technique.

Procedure = a real-world example of a hacker group using that technique.

Think of it like cooking:

[Tactics (the WHY — the course)] × [Techniques (the HOW — the recipe)]

Tactic = “make dessert,” Technique = “bake a chocolate cake,” Sub-technique = “use dark chocolate.”

Story Time — An Analogy to Remember

The Burglar's Night — The Same Steps, Every Time:

A burglar rarely just runs in and grabs. They follow a rhythm you can learn:

- **Reconnaissance** (looking around): walk past the house, check cameras, try door handles. On a network: scan ports, look up info.
- **Initial Access** (getting in): pick the back door. On a network: exploit a website bug, trick someone with a bad email.
- **Execution** (doing something inside): step inside. On a network: run a program on the hacked machine.
- **Persistence** (leaving a spare key): hide a key under the mat so you can come back. On a network: install a backdoor.
- **Privilege Escalation** (finding the master key): find the key that opens *every* room. On a network: become admin.
- **Defense Evasion** (avoiding cameras): wear a mask. On a network: hide in normal traffic.
- **Lateral Movement** (sneaking room to room): open every door inside. On a network: jump from one computer to another.
- **Collection** (gathering valuables): put jewellery in a bag. On a network: gather files.
- **Command and Control (C2)** (calling the boss): phone a friend outside for instructions. On a network: a hidden chat with the hacker's server.
- **Exfiltration** (carrying the bag out): leave with the loot. On a network: send data out.

Learn this rhythm, and you can predict the next move: "They just cased the house — next they will try a door."

3.1.1 The Full List of Tactics (In Order They Usually Happen)

1. **Reconnaissance** — gather info about the victim.
2. **Resource Development** — build tools / buy servers for the attack.
3. **Initial Access** — get a first foothold.
4. **Execution** — run code on the victim.
5. **Persistence** — stay even after reboot.
6. **Privilege Escalation** — get higher permissions.
7. **Defense Evasion** — hide from alarms.
8. **Credential Access** — steal passwords.
9. **Discovery** — learn the inside layout.
10. **Lateral Movement** — move to other machines.
11. **Collection** — gather interesting data.
12. **Command and Control (C2)** — talk to the hacker's server.
13. **Exfiltration** — steal data out.

14. **Impact** — destroy / disrupt something.

You do *not* need to memorise all 14 right now. For *this* project, focus on the five that are clearly visible as **network** traffic:

Key Idea — Remember This!

The five **network-visible** stages your World Model will predict:

Reconnaissance → **Initial Access** → **Lateral Movement** → **Command & Control** → **Exfiltration**

Your forecast is a *stage trajectory*: “In the next 5 windows, I think we go from Recon to Initial Access.” That is far more useful than just “attack / not attack.”

3.1.2 Some Real Technique IDs (So You Sound Like a Pro)

- **T1046 Network Service Discovery** (Recon): scanning which doors are open.
- **T1190 Exploit Public-Facing Application** (Initial Access): using a bug in a website to break in.
- **T1021 Remote Services** (Lateral Movement): using SSH or RDP (remote login) to jump to another computer.
- **T1071 Application Layer Protocol (C2)**: hiding hacker chat inside normal web / DNS traffic.
- **T1041 Exfiltration Over C2 Channel**: sneaking stolen data out through the same hidden chat.

The ID (like T1046) is just a page number in the encyclopaedia. Judges love when you say “We mapped predictions to ATT&CK T1046” instead of “We found scans.”

3.1.3 Why Mapping to ATT&CK Makes Your Project Stronger

Instead of teaching the AI to say “attack / normal” (a boring yes/no), you teach it to say “this is stage **X**”:

$$P(\text{stage}_{t+1\dots t+K} \mid \text{network seen so far})$$

Read this as: “The chance that the *next K stages* are this sequence, *given* what we’ve seen.” Now your demo can show a timeline with coloured stage labels — far more explainable and impressive.

Your Turn! — Try It Yourself (No Pressure)

Pick any heist movie you love (Ocean’s Eleven, Money Heist . . .). Map its steps onto the 14 tactics above. Which steps would a *network camera* see (e.g., phone calls = C2), and which happen *off-camera*? This is exactly the gap your AI must bridge by *predicting*.

3.2 2.2 Your Practice Papers: The Datasets

How do you *teach* an AI without giving it real attacks? You give it **datasets**: big saved recordings of real (or realistically simulated) network traffic, with answer keys.

Word Bank — New Word Alert!

Dataset = a saved collection of examples (flows/packets) + their correct labels (“benign” or “bot,” “attack stage = Recon,” etc.).

Benign = normal, harmless traffic (like regular visitors to the museum).

Malicious = traffic that is part of an attack.

Label = the answer key that says which is which.

Benchmark = a well-known dataset everyone uses, so you can fairly compare your AI to others (like a standard exam everyone takes).

3.2.1 The Two Datasets Named in Your Challenge

- **CIC-IDS 2017 / 2018 (Canadian Institute for Cybersecurity)** — The workhorse. They simulated a real office network with real web, mail, file transfers, and then unleashed many attacks: password guessing, flooding (DoS), website hacking, infiltration, botnets, port scans. They published both the raw PCAPs and, beautifully, **CSV files with 80+ pre-computed flow features**. That CSV is a gift — the report cards are already made for you.
- **CTU-13 (Czech Technical University)** — 13 separate stories, about 36 hours total, focused on **botnets** (networks of hacked machines controlled by a hacker). Each story has normal traffic + botnet traffic + background noise. Perfect to study that sneaky “Command and Control” heartbeat.

3.2.2 Other Datasets You Might Hear About

- **UNSW-NB15**: newer attacks like fuzzers, backdoors.
- **CSE-CIC-IDS2018**: like CIC-IDS but with full attack scenarios captured end-to-end.
- **NSL-KDD / KDD’99**: the grandpa of datasets — historically famous, but now too old and too easy. Good for learning, not for a serious modern prototype.

3.2.3 What One Row of CIC-IDS Looks Like (One Report Card)

One row = one flow (one conversation summary). Each column = one clue.

<i>Column Name</i>	<i>What it means (plain words)</i>	<i>Example</i>	<i>Usefulness</i>
Dst Port	Which door on the other house	80 (web)	high
Protocol	Which language (6=TCP, 17=UDP)	6	high
Flow Duration	How long the chat lasted (microseconds)	184097340	high
Flow Bytes/s	How fast data moved	730.4	high
Fwd Pkt Len Mean	Average size of your messages	58.0	high
Init Win bytes fwd	How big a desk the other side offered	29200	high
SYN Flag Count	How many “hellos” were sent	1	high
...	...	...	...
Label	Teacher’s answer: Benign or Bot?	Benign	target

Teacher’s Tip — For Your Project

Which dataset to start with? Start with the **CIC-IDS CSV**. It already has the 80+ features ready. You can skip packet parsing and jump straight to windowing (Chapter 1.10). Save PCAP parsing for when you want that extra packet-level detail or when you download raw captures. Hackathon time is precious — use the gift.

Your Turn! — Try It Yourself (No Pressure)

If you have the CIC-IDS CSV, load it with Python's pandas:

```
1 import pandas as pd
2 df = pd.read_csv("cic-ids-flows.csv")      # load the spreadsheet
3 print(df["Label"].value_counts())         # count: how many Benign vs
      Bot?
4 print(df.head())                          # peek at first 5 report
      cards
```

You will likely see: *way more Benign than Bot*. That **imbalance** matters a lot for how we measure success (Chapter 5). Note it down.

3.3 2.3 The Silent Cheater: Temporal Data Leakage

This is the most important warning in the whole book. Please read it slowly.

Story Time — An Analogy to Remember**The Cheating Student:**

A student secretly gets tomorrow's exam answers, practises on them all night, scores 100% in class, and everyone thinks they are a genius. But in the real exam with *new* questions, they fail. That is **data leakage**: when information from the *test (future)* sneaks into the *training (past)*, making your model look brilliant in practice but useless in real life.

In time-series security data, this is the **number one reason** students' prototypes look amazing on their laptop and flop in front of judges.

Word Bank — New Word Alert!

Data leakage = when your training data accidentally contains clues from the test data / future, so the AI cheats without you realising.

Train / test split = dividing your dataset into a "study set" (train) and a "secret exam" (test). The AI must NOT see the exam while studying.

Scaler / normalisation = rescaling numbers so they are all on a similar range (like converting all marks to percentages). If you learn the scaling from the test data, you peeked.

3.3.1 Four Ways Leakage Sneaks In (and How to Block Each)

1. **Random split** — You shuffle all windows and randomly pick 80% for train, 20% for test. But window 100 and window 101 are from the *same attack*, just seconds apart. Now the same attack is in both sets. *Fix*: split by **time**, not randomly.
2. **Scaling before splitting** — You rescale the whole dataset *before* splitting, so the scaler's average was computed from test data too. Peeking! *Fix*: learn scaling from train only, then *apply* it to test (in code: **fit** on train, **transform** on test).
3. **Feature that IS the answer** — A column that secretly encodes the label (like a column called "is_malicious" that is just the answer). *Fix*: drop suspicious columns, read dataset docs.
4. **Bleeding across windows** — When you build window summaries, you accidentally include packets from the *future* window in the past window (e.g., overlapping windows that leak). *Fix*: build windows strictly left-to-right in time.

3.3.2 The Honest Rules (Pin This to Your Wall)

1. **Split by time.** Example: first 70% of the timeline for train, next 15% for validation, last 15% for test. Keep whole attack stories together, never cut one story in half across the split.
2. **Fit scalers on train only.** Remember: `fit_transform` on train, `transform` only on test.
3. **Use walk-forward validation.** Train on weeks 1–2, test on week 3; then train on weeks 1–3, test on week 4 ... Like studying up to chapter 3, then testing on chapter 4, rolling forward.
4. **Be suspicious of 99% scores.** On imbalanced security data (99% benign), a cheating or broken model can hit 99% by doing almost nothing. That is a red flag, not a celebration.

Walk-Forward Validation — The Honest Way (with comments):

```

1  # Imagine X is your list of window-summaries, in time order.
2  # y is the label for each window (e.g., "Benign" or "Bot").
3  n_windows = len(X)
4
5  for fold in range(k_folds):
6      # 1. Define where train ends and validation begins FOR THIS FOLD
7      .
8      train_end = int(n_windows * (fold + 1) / (k_folds + 1))
9      val_end    = train_end + val_size
10
11     # 2. Cut: past only for training, immediate future for
12     validation.
13     Xtr, ytr = X[:train_end], y[:train_end]      # past
14     Xva, yva = X[train_end:val_end], y[train_end:val_end]  # future
15
16     # 3. Learn scaling from train ONLY, then apply to both.
17     scaler.fit(Xtr)                                # learn average from
18     past only!
19     Xtr = scaler.transform(Xtr)                    # rescale past
20     Xva = scaler.transform(Xva)                    # rescale future with
21     PAST's rule
22
23     # 4. Train and test this fold.
24     model.fit(Xtr, ytr)
25     evaluate(model, Xva, yva)                      # roll forward to next
26     fold

```

Notice: the model never sees the future while training. Honest!

Quick Recap — What We Just Learned

Chapter 2 recap: ATT&CK is the hacker's playbook (tactics=why, techniques=how). Your AI predicts the *stage* trajectory, not just yes/no. CIC-IDS and CTU-13 are your practice papers. And the golden rule: **never let the future leak into the past** — split by time, fit scalers on train only.

4. Chapter 3: Teaching AI to Imagine the Future

AI Dynamics & Sequence Models (The World Model)

This is the heart of your project. Take a deep breath — we will go slowly and draw many pictures in your mind.

4.1 3.1 What Is a World Model? A Weather Forecast for Hackers.

So far, most security tools are like a photo classifier: “Look at THIS one picture — is it a cat or a dog? Is it attack or normal?” Useful, but limited.

A **World Model** is like a **weather forecaster**. It doesn’t just say “Is it raining NOW?” It learns how weather *moves* — “clouds are drifting east, pressure is dropping, so it will probably rain in 2 hours.” It has an *internal simulation* of the world that it can run forward in its mind.

Key Idea — Remember This!

Classifier asks: “*What is this?*” (one snapshot)

World Model / Forecaster asks: “*What happens next — and how sure am I?*” (a movie)
Your challenge wants the **second**. You must forecast *attacker progression* over the next K windows (time slices).

4.1.1 The One Equation That Says It All (and What It Means)

$$P(S_{t+1} | S_t)$$

Read it as: “The **chance** (P) of the **next state** (S_{t+1}) *given* the **current state** (S_t).”

What is a “state”? Just a snapshot summary of the network at time t (the feature vector of that window). So the equation says: “Given how the network looks right NOW, what will it probably look like NEXT?”

If the World Model learns this well, it can dream forward: $S_t \rightarrow S_{t+1} \rightarrow S_{t+2} \rightarrow \dots \rightarrow S_{t+K}$. That dream is your forecast.

4.1.2 The Three Body Parts of a World Model

The famous “World Models” paper (Ha & Schmidhuber, 2018) describes three parts, like a body:

1. **Eyes (Vision / Encoder, called V):** Takes the raw messy picture and compresses it into a small, tidy code z_t (like a zip file for the network snapshot). “I see a lot, but I will remember the essence.”
2. **Brain (Memory / Dynamics, called M):** Learns how that code evolves: “Given z_t , what will z_{t+1} be?” It has a **hidden state** h_t — its memory of the past. This is usually an LSTM, Transformer, or GNN (we will meet all three).
3. **Mouth (Controller / Predictor, called C):** Turns the brain’s dream into a human answer: “I think the next stage is **C2** with 80% chance.”

For you: **V** = turns a window’s flows into a vector, **M** = learns how vectors follow each other, **C** = predicts the next stage / probability.

4.1.3 How We Train It (Without Cheating)

We show it many real recordings: “When the network looked like THIS at time t , it looked like THAT at time $t + 1$.” The correct next state is known from the dataset’s timeline. The model turns its knobs (weights) to make its prediction match the real next state, over and over.

Because attacks are rare and hackers keep inventing new tricks, we want the model to learn *the pattern of transition*, not to memorise one specific attack's bytes. Then it can handle *new* attacks it never saw before — that is the test of a true World Model.

Your Turn! — Try It Yourself (No Pressure)

In one sentence each: (1) What does a classifier do? (2) What does a World Model do? If you can answer, you have the most important idea of the whole course. Write them on a sticky note and keep them visible while you study Chapter 3!

4.2 3.2 Why Normal AI Forgets: And How We Fix It with Memory

4.2.1 The Problem with a Forgetful Friend

A normal neural network (feed-forward) is like a friend with no memory. You tell them today's news, they react, and instantly forget yesterday. But hacker attacks unfold over many windows — the scan happened 10 windows ago, the break-in now, the theft 5 windows from now. To connect those dots, the AI needs **memory**.

That memory is a **hidden state** h_t : a list of numbers the AI carries forward in time, updating it at each step. The family of models that do this is called **Recurrent Neural Networks (RNNs)**.

4.2.2 The Simple RNN: The Basic Memory Cell

At each new time step t , the RNN does:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

Let's read it in English, piece by piece:

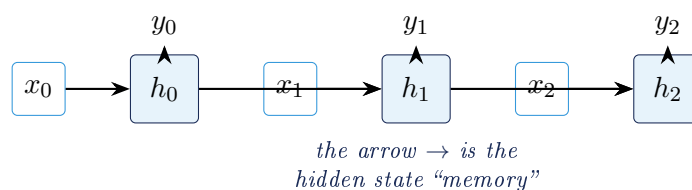
- x_t = what you see right now (the new window's features).
- h_{t-1} = what you remembered from before.
- W_{xh} and W_{hh} = how strongly to listen to the new info vs. the old memory (learned knobs).
- b_h = a small nudge (bias).
- $\tanh$ = squish the result to between -1 and 1 so numbers don't explode.
- h_t = the new memory you will carry to the next step.

And it also produces an output:

$$y_t = W_{hy} h_t + b_y$$

("Based on my current memory, here's my prediction.")

The SAME knobs W_{hh} , W_{xh} , W_{hy} are used at every time step — that sharing is how one small network can handle a movie of any length.



Think of it as passing a whispered note down a line of friends: each friend reads the new word (x_t), updates the note (h_t), and passes it on.

4.2.3 The Whisper Game Problem: Vanishing Gradients

Remember the game “telephone” where a whisper gets quieter and quieter down the line until the last person hears nothing? That is exactly what happens during training with a simple RNN, called the **vanishing gradient** problem.

To learn, the AI sends a “how wrong was I?” signal backwards through time (**Backpropagation Through Time**). Each step multiplies that signal by the hidden knob W_{hh} . If that knob is a bit less than 1, multiplying many times makes the signal *vanish* to almost zero — the early steps learn nothing. If the knob is a bit more than 1, the signal *explodes* to infinity. Either way, long-range memory fails. And multi-stage attacks NEED long memory.

Simple RNNs are bad at remembering something from 20 windows ago. We need a better notebook.

4.3 3.3 LSTM: The Smart Notebook with Gates (Doors)

The **Long Short-Term Memory (LSTM)**, invented in 1997, fixes the whisper problem with a brilliant idea: carry a separate **cell state** c_t that acts like a **conveyor belt** / **highway** through time, with little **gates (doors)** that control what gets on and off.

Imagine This — Picture It in Your Mind

Picture a factory conveyor belt (c_t) carrying a notebook.

- A **Forget Gate** is a door that can throw some pages away.
- An **Input Gate** is a door that can add new pages.
- An **Output Gate** is a door that decides what to read out loud to the world (h_t).

Because the belt mostly just *carries* the notebook with tiny edits (add some, erase some), the “how wrong was I” signal can travel far without vanishing. Long memory is saved!

Here are the six equations — I will tell you the story of each one. Don’t memorise symbols; understand the *job* each gate does.

Let x_t be the new window, h_{t-1} the previous read-out, and $[h_{t-1}, x_t]$ just means “glue them together into one long list.”

1. **Forget Gate** — “How much of the old notebook should I keep?”

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

σ squishes to 0–1. So f_t is a list of numbers like [0.9, 0.1, 0.8] meaning “keep 90% of page 1, only 10% of page 2 ...” 0 = forget fully, 1 = remember fully.

2. **Input Gate** — “How much new info should I let in?”

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Same idea: a 0–1 mask for how much of the new candidate to write.

3. **Candidate** — “What new stuff COULD I write?”

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

This is the *actual new content* proposed, squished to –1 to 1.

4. **Update the Notebook** — “Make the edit.”

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Keep some old ($f_t \odot c_{t-1}$) PLUS add some new ($i_t \odot \tilde{c}_t$). Remember $\odot$ means multiply matching items. This is the highway!

5. Output Gate — “What should I show the world?”

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

6. New Read-Out — “Read a cleaned-up version of the notebook aloud.”

$$h_t = o_t \odot \tanh(c_t)$$

Key Idea — Remember This!

If the equations feel heavy, just remember the **notebook story**: *Forget some pages, write some new pages, read some pages aloud*. That is all an LSTM does, 100 times per second. The cell c_t is the notebook, h_t is what you read aloud.

4.4 3.4 GRU: The Lighter Backpack

The **Gated Recurrent Unit (GRU)**, from 2014, is LSTM’s simpler sibling. It merges “forget” and “input” into one door called the **update gate**, and it uses the read-out h_t directly as the memory (no separate notebook). Fewer knobs, faster, often just as good.

- **Reset Gate** $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$: “How much old memory should I ignore when making a new draft?”
- **Update Gate** $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$: “How much of the new draft should I actually use?”
- **Candidate** $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$: the new draft (old memory first filtered by r_t).
- **Blend** $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$: Mix old and new. If z_t is near 0, keep the old. If near 1, adopt the new draft. Like fading between two photos.

Key Idea — Remember This!

Which should YOU use first? Start with **GRU or LSTM**. They are simple, well-tested, train fast on modest data, and beat fancier models surprisingly often. Save Transformers for when you have lots of data and want long-range spotlight attention.

Your Turn! — Try It Yourself (No Pressure)

Cover the “meaning” column for the LSTM gates and try to explain each gate’s job in your own words to an imaginary 12-year-old. If you can do it without peeking, you own it. Then answer: why is c_t called a “highway”?

4.5 3.5 Transformers and Self-Attention: The Spotlight

4.5.1 Why Another Model If LSTMs Work?

LSTMs read time like a book: page 1, then page 2, then page 3, one by one. That is slow (can’t parallelise easily) and the link between page 1 and page 100 is still a long chain.

Transformers read the WHOLE book *at once* and use a trick called **self-attention** to let any page directly look at any other page. The distance between page 1 and page 100 is one hop, not 99 hops. That is powerful for spotting a scan 20 windows ago that relates to a theft now.

4.5.2 The Spotlight Analogy

Imagine This — Picture It in Your Mind

Imagine a classroom. The teacher asks: “Who knows the answer?” Each student (student = one time window) has:

- A **Question** they are asking: “I need info about scans.” This is the **Query** (q).
- A **Label** on their desk: “I have scan info / I have exfil info.” This is the **Key** (k).
- **Knowledge** in their head: the actual info they know. This is the **Value** (v).

When student i wants to answer, they compare their Question (q_i) to every other student’s Label (k_j). The better the match, the brighter the **spotlight** (α_{ij}) shines on that other student’s Knowledge (v_j). The final answer for student i is a weighted mix of everyone’s knowledge, with brighter spotlights counting more.

That, literally, is self-attention.

In maths:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

$$\text{score}(i, j) = \frac{q_i^\top k_j}{\sqrt{d_k}} \quad (\text{how well } i\text{'s question matches } j\text{'s label})$$

$$\alpha_{ij} = \text{softmax}_j(\text{score}(i, j)) \quad (\text{turn scores into spotlight brightnesses that sum to 1})$$

$$\text{out}_i = \sum_j \alpha_{ij} v_j \quad (\text{mix everyone's knowledge by spotlight})$$

The $\sqrt{d_k}$ just stops numbers from getting too huge and confusing the softmax (like dimming a blinding spotlight to a comfortable level).

A sentence explains it best:

“The cat chased the dog because *it* was scared.”

Who is “it”? Self-attention lets “it” shine its spotlight on “dog” more than on “cat.” In network terms, it lets the model decide: “This exfiltration spike at window 50 is related to that SYN burst at window 30, not to the normal web traffic at window 49.”

4.5.3 Multiple Spotlights and Page Numbers

- **Multi-head attention** = run several spotlights in parallel, each with its OWN W_Q, W_K, W_V . One head might learn to watch *ports*, another to watch *timing*. Then glue their answers together. More eyes, more patterns.
- **Positional encoding** = Because attention looks at all pages at once, it has *no idea about order* by itself. So we add a tiny number pattern (a “page number” written in invisible ink) to each input:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Modern time-series Transformers sometimes use fancier clocks (“time2vec”) that also encode time-of-day or periodicity.

4.5.4 The Transformer Block = LEGO Brick

One block = **Multi-Head Attention** → add & normalise → **Feed-Forward** (a small normal network) → add & normalise. Stack several blocks. For forecasting, feed H past windows, predict the next K .

Common Mistake — Don't Do This!

Attention alone is NOT a prediction. It is just the spotlight. You still need a *task head*: a small final layer that turns the spotlighted mix into an actual answer (e.g., “stage = C2 with 90%”), plus a *loss* to train it (like cross-entropy for “which stage?”). Don't mistake the spotlight for the whole torch!

4.6 3.6 Dreaming K Steps Ahead: The Rollout

Now put it together. Given today's snapshot S_t , the **prediction engine**:

1. **Encodes** the now.
2. **Dreams** forward K steps: predicts S_{t+1} , feeds that prediction back in as if it were real, predicts S_{t+2} , and so on up to S_{t+K} . Like writing a story one sentence at a time, using your own last sentence as the prompt for the next.
3. **Collects** the dream into three human outputs:
 - a. **Probability timeline**: how likely is trouble in each of the next K windows? Example: [0.1, 0.3, 0.7, 0.85, 0.9] (getting worse).
 - b. **Stage trajectory**: which ATT&CK stage at each step? Example: Recon → Recon → Initial Access → C2.
 - c. **Reasons**: why does the model think so? (SHAP + attention, next chapter).

4.6.1 Teacher Forcing vs. Real Dreaming

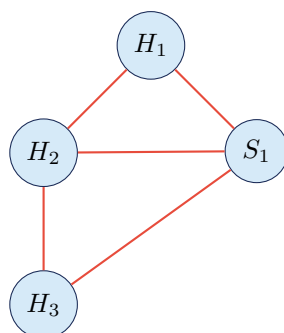
- **Teacher forcing (training)**: Feed the *real* next step as input, even if the model's own prediction was wrong. Like a teacher whispering the correct next word during practice. Learns fast, but never practises recovering from its own mistakes.
- **Autoregressive rollout (real life)**: Feed the model's *own* last prediction as the next input. This is how it will actually run, but mistakes can *compound*: a small error at step 1 becomes a bigger error at step 5. We often *mix* both during training (called **scheduled sampling**) so the model learns to be robust.

Common Mistake — Don't Do This!

Because errors stack, keep K *modest* (maybe 5–10 windows), report scores *per step* (“step-1 accuracy, step-5 accuracy”) not just one number, and show how confidence fades further into the future. Honesty builds trust.

4.7 3.7 Graph Neural Networks: The Social Network of Computers

So far, we treated windows as lists. But a network is really a **social network**: computers are *people* (nodes), and flows are *friendships* (edges). That shape matters! A scanner talks to *many* strangers (one-to-many). A server that is being DDoS'd is talked to *by many* people (many-to-one). That shape IS a clue.



4.7.1 Gossip as Maths: Message Passing

How do GNNs work? Like **gossip**. Each person (node) listens to their neighbours, mixes what they hear with what they already know, and updates their own story. Do this layer by layer, and after a few rounds each person knows a bit about the whole town.

For node v at layer ℓ :

$$h_v^{(\ell+1)} = \text{UPDATE}\left(h_v^{(\ell)}, \text{AGGREGATE}(\{h_u^{(\ell)} : u \in \mathcal{N}(v)\})\right)$$

In words: “My new story = combine (my old story, the combined story of all my neighbours).” AGGREGATE is usually an average or a sum. UPDATE is a small neural net.

Popular flavours:

- **GCN** (Graph Convolutional Network): neighbour average, with a neat normalisation.
- **GAT** (Graph Attention Network): neighbours don’t count equally; the model learns *who to listen to more* (attention on the graph).
- **GraphSAGE**: samples a few neighbours so it scales to huge towns.
- **GIN**: very expressive for telling different shapes apart.

4.7.2 GCN in One Line (Don’t Fear It)

$$h_v^{(\ell+1)} = \sigma\left(W^{(\ell)} \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\deg(u) \deg(v)}} h_u^{(\ell)}\right)$$

Translation: “Average my neighbours’ stories and my own, but divide by how popular each person is so a super-popular person doesn’t dominate. Then turn the knob W and squish with σ .” That division is just to keep numbers well-behaved.

4.7.3 Time + Graph = Spatio-Temporal

Because we have time *and* a graph, we combine them:

- **ST-GCN**: do a graph gossip step, then a time step (1D convolution), repeat. Like alternating between “who talks to whom” and “how things change over time.”
- **GNN + LSTM/Transformer**: GNN makes a good summary of the *town shape* at each time slice; then an LSTM/Transformer watches the *movie* of those summaries.

Teacher’s Tip — For Your Project

Where to start? Don’t build a huge graph on day one. For your first prototype, build a simple **host graph** per window (nodes = computers, edges = flows seen in that window), run a tiny GCN to get a shape summary, and feed that sequence into a GRU. Simple, explainable, and judges will love that you thought about topology.

Quick Recap — What We Just Learned

Chapter 3 recap: A World Model is a *forecaster*, not just a classifier: it learns $P(S_{t+1} | S_t)$ and dreams K steps ahead. LSTMs carry a notebook with gates to remember far back; GRUs are the lighter version; Transformers use spotlights (attention) to connect far-apart pages; and GNNs add the social-network shape via gossip (message passing). Layer them to build your dream machine.

5. Chapter 4: Making AI Explain Itself

Interpretability & Explainable AI (XAI)

5.1 4.1 Why “Trust Me, I’m AI” Is Not Enough

Your challenge says, in bold: “*Black-box models are unacceptable.*” You **MUST** include an **Explainability Engine** that turns “the AI said so” into “here is *why* the AI said so, in words a human can check.”

Why? Imagine a doctor says “You need surgery” but refuses to say why. Would you trust them? A security analyst feels the same about an AI that just shouts “ATTACK!” with no reason. Explainability builds **trust**, lets you **debug** mistakes, and impresses judges.

Two questions matter:

1. “**Which clues (features) made the difference?**” Was it the many SYNs? The weird port? The fast timing? → Answer: **SHAP**.
2. “**Which time slices did the model stare at?**” Did it look at the scan 20 windows ago or just the last one? → Answer: **attention weights**.

One tells you *what* mattered, the other *when/where*.

5.2 4.2 SHAP: Fairly Sharing Credit (The Pizza Party)

5.2.1 The Pizza Party That Explains Everything

Story Time — An Analogy to Remember

Five friends order a pizza together. The bill is \$50. But the fun they had depends on *who came*. Alice alone is quiet, but Alice + Bob together are hilarious. How do you fairly say “Alice contributed this much to the fun, Bob this much . . .” when the value depends on the *group*?

Lloyd Shapley, a mathematician, proved there is ONE fair way: for each person, look at *every possible group* they could join, see how much the group’s fun *increases* when they join, and average that over all groups. That average increase is their **Shapley value**.

In AI: the “friends” are your **features**, the “fun” is the **prediction** (e.g., 0.92 = “probably an attack”). SHAP does the same fair averaging to say “Feature ‘many SYNs’ pushed the prediction up by +0.25.”

Key Idea — Remember This!

Shapley value = a feature’s *average* extra contribution to the prediction, averaged over every possible combination of other features. SHAP makes this practical with a clever *additive* trick so you don’t have to try literally every combination (which would take forever).

5.2.2 The Additive Sentence

SHAP writes the explanation as a simple sum:

$$g(x') = \phi_0 + \sum_{i=1}^M \phi_i x'_i$$

In English:

- x'_i is 1 if feature i is “present,” 0 if “absent” (like “is Alice at the party?” yes/no).

- ϕ_0 is the **baseline**: the average prediction over all examples (like “on a random day, the AI says 10% chance of attack”).
- ϕ_i is the **SHAP value** for feature i : how much that feature pushed the prediction away from baseline for *this* specific example.
- $g(x')$ should be close to the real model output $f(x)$ for this example (local accuracy).

5.2.3 How to Read a SHAP Value (Like Reading a Report Card)

- **Positive ϕ_i** (e.g., +0.30) = this feature pushed the answer *UP* toward “attack.”
- **Negative ϕ_i** (e.g., -0.20) = this feature pulled it *DOWN* toward “benign.”
- **Big absolute value** = very important for this decision.
- **Small near zero** = didn’t matter much this time.

A real reading:

The AI says “attack with 92% chance.” The average (baseline) is only 10%. So it moved +0.82 above baseline. Why?

$$\phi(\text{weird dst port}) = +0.30, \quad \phi(\text{high SYN count}) = +0.25$$

$$\phi(\text{very short gaps between packets}) = +0.15$$

Now a human analyst reads: “Ah, it flagged because of the port, the many hellos, and the burstiness. That matches a scan.” *Explainable!*

5.2.4 SHAP in Python (You Don’t Need to Code This From Scratch)

```

1 import shap
2
3 # 1. Make an explainer for your trained model.
4 explainer = shap.TreeExplainer(model)      # for tree models; exact and
      fast
5
6 # 2. Ask it to explain some examples.
7 shap_values = explainer.shap_values(X_sample)
8
9 # 3. Draw pictures.
10 shap.summary_plot(shap_values, X_sample)    # which features matter
      overall?
11 shap.force_plot(explainer.expected_value, shap_values[0], X_sample.iloc
      [0])
12                                     # why this ONE example?
```

Which explainer to pick?

- **TreeExplainer** — for tree models (Random Forest, XGBoost). Exact, fast. Great for baselines.
- **Gradient / DeepExplainer** — for neural nets (LSTM, Transformer). Uses gradients (the learning signal).
- **KernelExplainer** — works with ANY model but is slow; use on a few examples only.
- **LinearExplainer** — for simple linear models.

Common Mistake — Don't Do This!

SHAP is **per-example** and **model-specific**. For your sequence model, explain the *final input window's feature vector* (the thing that went into the last prediction head), not the raw packets. And always remember: SHAP explains what the *model* did, not necessarily what is true in the real world.

5.3 4.3 Attention Weights: Peeking Over the AI's Shoulder**5.3.1 What Are They?**

In a Transformer, the spotlight brightnesses α_{ij} say: “When making the answer for position i , how much did I look at position j ?” If you save those numbers, you can literally see *where the model was looking*.

Key Idea — Remember This!

SHAP = “*which clues* mattered” (the what).

Attention = “*which time slices* the model stared at” (the when).

Use both. They answer different questions, and judges love seeing both in a demo.

5.3.2 How to Get Them in Code

During the forward pass (the prediction), the Transformer computes attention inside. You just save it:

```
1 # In PyTorch, after a forward pass:
2 attn_weights = model.get_attention(input_window) # shape: [heads, time
3 # attn_weights[h, i, j] = how much head h, at time i, looked at time j
```

In many libraries you can add a **hook** (a little spy) that saves the tensor without changing training code at all.

5.3.3 The Heatmap (The Picture Judges Remember)

Make an $n \times n$ coloured grid: rows = “who is asking” (query time), columns = “who is being looked at” (key time), colour = brightness.

What you usually see:

- A bright **diagonal**: each moment mostly looks at itself and neighbours (natural).
- Occasional bright **off-diagonal spots**: a far-away moment that mattered a lot (e.g., a scan 20 windows ago suddenly relevant now). Those spots are the *long-range connections* your World Model learned — point them out in your demo!

Common Mistake — Don't Do This!

Research warns: attention is a *useful signal*, not a *proof*. A model can attend to something without truly using it, or use something without attending strongly. Treat attention as *evidence*, combine it with SHAP, and you will be honest and strong.

Teacher's Tip — For Your Project

Demo gold: On one screen, show: (1) the probability timeline going up, (2) the predicted ATT&CK colours per window, (3) the attention heatmap with circles on the key spots, and (4) a SHAP bar chart saying “flagged because: many SYNs, weird port, burst timing.” That is end-to-end explainability. Judges will applaud.

Quick Recap — What We Just Learned

Chapter 4 recap: Black boxes are not allowed. SHAP fairly shares credit among features (like sharing pizza bill fairly). Attention shows where in time the model looked. Together they turn “Trust me” into “Here’s why, check it yourself.”

6. Chapter 5: Turning It Into a Real Product

Deployment, Benchmarking & Systems

So you built a World Model that can dream. How do you turn that lab experiment into a *real thing* someone can download, run offline, and trust? And how do you *prove* it is better than the simple thing?

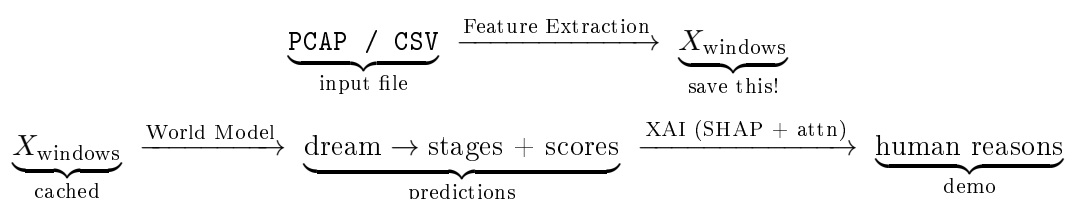
6.1 5.1 Building an Offline Product (No Internet Needed)

Your challenge requires a **fully open-source, fully offline** product. No calling ChatGPT, no cloud. Everything must run on the judge's laptop with no Wi-Fi. That shapes every choice.

6.1.1 The Six Pieces You Must Deliver (The Check-list)

1. **Feature Extraction Pipeline** — the code that turns a raw PCAP or CSV into the clean, normalised table of numbers (the feature matrix) with timestamps. This is the kitchen that prepares ingredients.
2. **Trained World Model** — your actual AI: training scripts, saved knobs (weights), and a config file so anyone can reproduce your training exactly.
3. **Prediction Engine** — the code that takes a fresh snapshot S_t and *dreams* K steps: $S_{t+1} \dots S_{t+K}$ plus probabilities and stages.
4. **Explainability Engine** — the code that runs SHAP and grabs attention maps to list *driving features* in human words.
5. **Demo Interface** — an **offline** app a judge can double click: maybe **Streamlit** (pretty web app), **Flask**, or even a simple **CLI** (text menu). It must: accept a PCAP/CSV, run the whole pipeline, and show the timeline, flagged flows, and stage colours. No internet.
6. **Benchmark Results** — a table proving you beat a simple **logistic regression** on the *same* features. Without this, your claim is just words.

6.1.2 The Assembly Line (How Data Flows Through Your Product)



Keep each arrow as a separate function/class so you can test and swap pieces without breaking everything (like LEGO, not like a tangled knot of wires).

Teacher's Tip — For Your Project

Speed matters in a live demo. Parsing a big PCAP can take minutes. **Cache** (save) the feature matrix to a file after the first run so the second run is instant. Also log the dataset filename and a hash (like `sha256`) so you can prove “this demo ran on exactly this file, these features.”

6.2 5.2 How Do We Know It's Good? Measuring with Honest Numbers

6.2.1 Your Opponent: The Simple Baseline

The challenge says you must beat:

baseline = logistic regression trained on the SAME features

What is **logistic regression**? Just the simplest sensible AI: draw a straight line (well, a gentle S-curve) to separate benign from attack. If your fancy World Model can't beat this simple line, something is wrong.

Same features is the key: both models must see the *same* windowed table. Otherwise you are not comparing AIs, you are comparing kitchens.

6.2.2 The Four Numbers That Matter (Meet the Confusion Matrix)

Forget “accuracy” for a minute. On security data it lies (we will see why). Instead, imagine we test on 100 examples and count:

	AI says Attack	AI says Benign
Really Attack	TP (True Positive) = caught	FN (False Negative) = missed
Really Benign	FP (False Positive) = false alarm	TN (True Negative) = correctly ignored

Now the four scores, with a tiny example: Suppose TP=80, FN=20, FP=10, TN=90.

- **Precision** = $\frac{TP}{TP + FP} = \frac{80}{80 + 10} = 0.89$:

“When my alarm rang, how often was it REAL?” High precision = few false alarms. Low precision = you cry wolf and analysts stop listening.

- **Recall** = $\frac{TP}{TP + FN} = \frac{80}{80 + 20} = 0.80$:

“Of all real attacks, how many did I catch?” High recall = you rarely miss a burglar. Low recall = burglars slip through.

- **F1** = $2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$:

The harmonic mean (a balanced blend) of the two. If either is low, F1 is low. With 0.89 and 0.80, $F1 \approx 0.84$.

- **False Positive Rate (FPR)** = $\frac{FP}{FP + TN} = \frac{10}{10 + 90} = 0.10$:

“How often do I falsely accuse innocent traffic?” In security, high FPR is deadly — analysts burn out and ignore your tool.

Common Mistake — Don't Do This!

Why accuracy fools you: Imagine 99 of 100 examples are benign, 1 is attack. A lazy AI that always says “benign” gets accuracy = 99/100 = 99%! It looks amazing but caught ZERO attacks. Precision/recall/F1 expose this trick. On imbalanced data, **never lead with accuracy**.

6.2.3 Extra Scores for Forecasting (You Are a Forecaster, Remember)

- **Per-step scores:** F1 at step 1, step 2, ... step K. Step-1 is usually high, step-K fades. Showing the fade is honest and expected.
- **AUC-ROC / AUC-PR:** how well you *rank* examples without picking a threshold. AUC-PR is better when attacks are rare (imbalanced).
- **Calibration / Brier score:** “When you say 80% chance, does it actually happen 80% of the time?” Good calibration means your percentages are trustworthy.
- **Stage-accuracy:** “How often did you get the ATT&CK stage right?” (Recon vs. C2 etc.).

6.2.4 The Honest Benchmark Recipe (Do This Exactly)

1. **Split by time**, leakage-free (Chapter 2.3).
2. **Fit scalars on train only** (Chapter 2.3).
3. Train **both** the simple baseline AND your World Model on the *identical* feature table.
4. Test both on the *same* held-out future timeline.
5. Report precision, recall, F1, FPR *per horizon step* (not one blurry number).
6. Sanity-check with a **random guesser** and a **plain LSTM** so you know your gain is real, not luck.

How your benchmark table should look:

Model	Precision	Recall	F1	FPR
Random Guessing	0.50	0.50	0.50	0.50
Logistic Regression (required baseline)	0.78	0.62	0.69	0.05
LSTM World Model (K=5)	0.86	0.79	0.82	0.03
Temporal Transformer	0.90	0.85	0.87	0.02
GNN + GRU	0.88	0.86	0.87	0.02

On one dataset these are point estimates. Use time-fold cross-validation and report ranges if you can. And say which metric you optimised for (often F1 or AUC-PR, not accuracy).

6.3 5.3 The Full Journey: Your 10-Step Roadmap

Tape this to your wall:

1. **Ingest** — load CIC-IDS CSV (or parse PCAP with Scapy).
2. **Make features** — flow + packet clues (Chapter 1.6).
3. **Slice time** — cut into H -length window histories, label each window (Chapter 1.10).
4. **Split honestly** — by time, no leakage (Chapter 2.3).
5. **Train the World Model** — GRU/LSTM/Transformer/GNN+GRU; pick your encoder + dynamics head; walk-forward validate.
6. **Dream forward** — K-step rollout → scores + stage path (Chapter 3.6).
7. **Explain** — attention + SHAP → driving features (Chapter 4).

8. **Build the demo** — Streamlit/Flask/CLI, offline, drag-and-drop PCAP/CSV → timeline + stages + attention + SHAP.
9. **Benchmark** — vs. logistic regression, with honest numbers (Chapter 5.2).
10. **Package** — source code on GitHub, README with setup steps, 2-page architecture doc, <2 minute demo video, <5 slides.

Key Idea — Remember This!

The One Sentence to Remember Forever:

A World Model is not a classifier that says “attack / not attack.” It is a *simulator of the network through time*:

See the state → *Learn* how it changes → *Dream* K steps ahead → *Name* the stage (ATT&CK) → *Explain* why (SHAP + attention) → *Prove* you’re better than simple (benchmark).

Master this loop and you haven’t just built a model — you’ve built a *radar that sees the future*. That’s a superpower.

Quick Recap — What We Just Learned

Chapter 5 recap: Build offline and open-source. Prove you beat a simple line (logistic regression) with precision/recall/F1 (not accuracy). Keep every stage as LEGO bricks. Cache slow steps. And make your demo tell *the whole story*: timeline + stages + spotlight + reasons.

A Final Word From Your Teacher — You Did It!

Take a breath. Look how far you came.

On page 1, “network telemetry” might have sounded like a foreign language. Now you know it is just “measuring the letters on the roads and writing report cards.” On page 1, LSTM might have sounded scary. Now you see a little notebook with three doors. That is real learning.

Here is the map of where you have been:

- **Chapter 0** gave you the ABCs: network = town, data = letters, AI = a baker who tastes and adjusts, P = chance.
- **Chapter 1** taught you how letters travel (IP = house, port = door, TCP = polite call with SYN/ACK), how we summarise them into flows, how we cut time into windows, and how to read a PCAP line by line.
- **Chapter 2** taught you how burglars think (ATT&CK = the playbook), where to get practice papers (CIC-IDS, CTU-13), and the golden rule: *never let the future leak into the past*.
- **Chapter 3** taught you to make AI *dream*: LSTMs with notebooks, GRUs as lighter backpacks, Transformers as spotlights, GNNs as gossip, and rollouts as dreaming K steps ahead.
- **Chapter 4** taught you to make AI *explain*: SHAP = fair pizza sharing, attention = where the spotlight shone.
- **Chapter 5** taught you to turn it into a real offline product and to prove it with honest numbers (precision/recall/F1, not accuracy).

A few last encouragements, from one builder to another:

- **Start simple.** A working GRU beats a broken 12-layer Transformer. Get one thing working, *then* climb.
- **Trust honest numbers.** If your score looks like 99.9%, check for leakage or imbalance first, not champagne. Honest 85% beats cheating 99% in front of a judge.
- **Explain everything.** A modest model that can say “because many SYNs on port 4444” beats a perfect black box that just says “attack.” Judges trust what they can see.
- **Show the movie, not just the snapshot.** Your demo should let a judge *watch* the attack unfold stage by stage, with the attention spotlight and SHAP reasons side by side.
- **Have fun.** You are building something that could one day help a real security team stop a real burglar. That’s a beautiful, meaningful thing to do. Be proud of it.

If you understood even 70% of this book on the first read, you are ahead of most beginners. Reread the chapters you found hard, do the “Your Turn” exercises with a friend, and run the code snippets on a small dataset. The second read will feel like a different book — an easy one.

Go build your World Model. I’m cheering for you, every step of the way.

— Your Teacher :)

P.S. If a line still confuses you, read it out loud as if you are explaining it to a 10-year-old. If you can explain it simply, you understand it deeply. That’s the Feynman trick — and it works.

Appendix A: Glossary — Every New Word in One Place

- **Accuracy** — fraction of all predictions that were correct. Lies on imbalanced data.
- **ATT&CK** — the playbook of hacker tactics and techniques (MITRE).
- **Attention** — spotlight that lets one time step focus on another.
- **Benign** — normal, harmless traffic.
- **Bias** (b) — a small nudge added in a neural net layer.
- **C2 (Command and Control)** — hacker secretly talking to their server.
- **Cell state** (c_t) — the notebook carried through time in an LSTM.
- **CIC-IDS / CTU-13** — well-known practice datasets with labelled traffic.
- **Classifier** — AI that says “what is this?” (one snapshot).
- **Dataset** — folder of examples + answer keys.
- **Discretisation** — turning smooth time into counted buckets.
- **Encapsulation** — wrapping data layer by layer, like nested boxes.
- **Exfiltration** — stealing data out.
- **Feature** — one clue (number) the AI reads.
- **Feature vector** — the list of all clues for one example.
- **Flow** — summary (report card) of one conversation (same 5-tuple).
- **Flow record** — one row in the flow table.
- **FPR** — False Positive Rate: how often you falsely blame innocent traffic.
- **GAT / GCN / GNN** — Graph Neural Network family; learns from town shape.
- **GRU** — lighter LSTM with two gates (reset + update).
- **Hidden state** (h_t) — AI’s memory carried through time.
- **IP address** — house address of a computer.
- **IPFIX / NetFlow** — standard ways to write and export flow report cards.
- **Label** — the correct answer for training (e.g., Benign/Bot).
- **Leakage** — when future/test info sneaks into training (cheating).
- **LSTM** — memory network with forget/input/output gates and a cell notebook.
- **PCAP** — file of saved packets (box of saved letters).
- **Port** — door number inside a house (e.g., 80 = web, 22 = SSH).
- **Precision** — when you said “attack,” how often were you right?
- **Protocol** — language inside (6=TCP, 17=UDP).
- **Recall** — of all real attacks, how many did you catch?
- **RNN** — Recurrent Neural Network; has memory over time.
- **Rollout** — dreaming K steps ahead, feeding each prediction back in.
- **Scaler** — rescales numbers to a similar range (e.g., 0–1).
- **SHAP** — fair way to share credit among features for one prediction.
- **SYN / ACK / FIN / RST** — TCP verbs: hello / got it / goodbye / abort.

- **Telemetry** — measuring from far away (here: measuring the network).
- **Transformer** — AI that reads all time steps at once with attention.
- **TTL** — countdown before a packet is thrown away (hops left).
- **Vector** — ordered list of numbers.
- **Weight (W)** — knob the AI turns to say how strongly to listen to a clue.
- **Window** — one time slice (e.g., 60 seconds) summarised into a snapshot.
- **World Model** — AI that learns how the world changes and dreams the future.
- **XAI** — Explainable AI; making the AI say *why* it decided.

Appendix B: Quick Reference — Key Equations

RNN (the basic memory)

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

LSTM — the six-gate notebook (jobs on the right)

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) & \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) & o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t & h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

GRU — the lighter backpack

$$\begin{aligned} r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r) & z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z) \\ \tilde{h}_t &= \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) & h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

Self-Attention (the spotlight)

$$\text{out}_i = \text{softmax}_j \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right) v_j$$

GCN (gossip on the graph)

$$h_v^{(\ell+1)} = \sigma \left(W^{(\ell)} \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{h_u^{(\ell)}}{\sqrt{\deg(u) \deg(v)}} \right)$$

SHAP additive explanation

$$g(x') = \phi_0 + \sum_{i=1}^M \phi_i x'_i$$

World Model dynamics

$$P(S_{t+1} \mid S_t)$$

Metrics

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

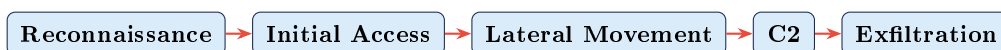
$$F1 = \frac{2PR}{P + R},$$

$$FPR = \frac{FP}{FP + TN}$$

Flow Feature Checklist (What to Compute Per Flow)

- 5-tuple: src/dst IP, src/dst port, protocol.
- Byte / packet counts (total, forward, reverse).
- Duration; flow bytes/s; flow packets/s.
- IAT stats: min, max, mean, std (forward + backward).
- Packet-size stats: min, max, mean, std.
- TCP flag counts (SYN, ACK, PSH, RST, FIN).
- Active / idle time stats.
- Packet-level extras: TTL variance, window size, fragment flags, retransmissions.

Appendix C: The Five Forecast Stages



This is the trajectory your World Model predicts over the next K windows. A good demo colours each window's prediction with one of these five, plus "Benign."